CS683 PA2 ��� Task 1: Stride Pattern Prefetching (SPP) at the L1D

- CS683 PA2 — Task 1: Stride Pattern Prefetching (SPP) at the L1D
  - 0. How this document maps onto the PDF’s deliverables
  - 1. Task 1A — what the supplied IP-stride prefetcher actually does
  - 2. SPP structure
  - 3. Dual pattern classification
  - 4. The signature — representing a *sequence* of strides
  - 5. Degree > 1: the lookahead chain
  - 6. Storage overhead — exact bit-level breakdown
  - 7. Accuracy-based throttling
  - 8. Results
  - 8.7 What the numbers actually say
  - 9. Toolchain finding — why the README’s GCC 7 section is load-bearing
  - 10. Consistency audit — does satisfying one requirement break another?

CS683 PA2 — Task 1: Stride Pattern Prefetching (SPP) at the L1D

> **Toolchain of record.** Every result in §8 was produced with **GCC 7.5.0 at `-O3`** (Ubuntu 18.04), as `README.md` requires, with **25M warmup + 25M simulation** instructions on all three supplied traces. The compiler is verifiable from the binaries themselves: `readelf -p .comment bin/<binary>` prints `GCC: (Ubuntu/Linaro 7.5.0-3ubuntu1~18.04) 7.5.0`. No number in §8 comes from any other toolchain. §9 explains why this matters more than it sounds.
>
> The only measurements in this document *not* from that toolchain are the two **design-selection sweeps** in §2.1 and §7.2, which were run at `-O0` purely to choose a table geometry and a degree cap before committing simulation time. They are labelled where they appear and none of them is reported as a result.

---

0. How this document maps onto the PDF’s deliverables

PDF deliverable (Task 1)	Section here
Task 1A — understand `prefetcher/ip_stride.l1d_pref`	§1
Task 1B — implement SPP	§2–§5, code in `prefetcher/spp.l1d_pref`
IPC, speedup, L1D load MPKI, accuracy, coverage — table + plots	§8
Total storage overhead, exact bit-level breakdown per entry	§6
How did you decide the accuracy-throttling threshold?	§7.1
Why these prefetch degrees for those accuracy ranges?	§7.2

Hard constraints the PDF imposes, and where each is satisfied:

Constraint	Where
Table tracks accesses per individual IP	`ip_table`, §2.1
Dual classification: uniform **and** non-uniform repeating strides	§3
Uniform first, dynamic fall-back to non-uniform	`uniform_conf`, §3.3
A **signature** of recent strides mapping to the next expected stride	`pattern_table`, §4
Degree **> 1**	§5, `MAX_DEGREE = 4`
Accuracy checker + dynamic degree scaling	§7
Never cross the OS page boundary	§2.2 — structural, not a post-hoc test
Lightweight / minimal storage	§6 — 2.56 KB total
Use the provided template file	written into `prefetcher/spp.l1d_pref`
25M warmup + 25M simulation	`scripts/run_task1.sh`

---

1. Task 1A — what the supplied IP-stride prefetcher actually does

`prefetcher/ip_stride.l1d_pref` is 139 lines. Stripped to its skeleton:

```
#define IP_TRACKER_COUNT 24
#define PREFETCH_DEGREE   3
#define MASK_IP        0xFF

class IP_TRACKER { uint64_t ip; uint64_t last_cl_addr; int64_t last_stride; uint32_t lru; };
IP_TRACKER trackers[IP_TRACKER_COUNT];
```

On every L1D access it:

1. Converts the address to a cache-line address: `cl_addr = addr >> LOG2_BLOCK_SIZE`. Strides are counted in **cache lines**, not bytes — prefetching is a line-granular operation, so this is the natural unit, and I kept it.
2. Masks the IP down to **8 bits** (`ip & 0xFF`) and does a **linear search** over 24 trackers for a match.
3. On a miss, it evicts the LRU tracker (the one with `lru == 23`), seeds it with the current address, and **returns without predicting** — one access carries no stride.
4. On a hit, it computes `stride = cl_addr - last_cl_addr` with an explicit signed-difference dance (the comment in the file explains it: subtracting `uint64_t`s and letting it wrap would produce a garbage magnitude).
5. **It prefetches only if `stride == last_stride`** — i.e. it needs to have seen the same stride twice in a row.
6. If so, it issues `PREFETCH_DEGREE = 3` prefetches at `cl_addr + k*stride`, `k = 1..3`, breaking out of the loop at the first address that leaves the 4 KB page.
7. It picks the fill level from MSHR pressure: `FILL_L1` while the L1D MSHR is less than half full, otherwise `FILL_L2`.

1.1 The five weaknesses I took as my design brief

Reading it this way makes the gaps precise, and each gap maps to one thing I changed:

#	Weakness in `ip_stride`	What it costs	My change
1	**One stride per IP.** The predictor's entire state is `last_stride`. The condition `stride == last_stride` is false on *every* access of a non-uniform pattern like `(+1,+2,+2,+2,+3)^n` except inside the run of `+2`s.	It prefetches on 2 of 5 accesses of the PDF's own example, and the addresses it emits (`+2,+4,+6`) are wrong for 2 of those.	Signature → next-stride table (§4)
2	**`ip & 0xFF` is the whole identity.** Any two load instructions whose PCs agree in the low 8 bits share a tracker and corrupt each other's history.	Silent cross-talk; a hot loop with loads 256 bytes apart in the binary destroys itself.	64 sets × 4 ways with a 16-bit tag → 22 bits of PC distinguished (§2.1)
3	**Fixed degree 3, always.** No notion of whether the prefetches are being used.	Pure pollution in phases with no stride structure, and under-prefetching in phases with perfect structure.	Accuracy-throttled degree 1–4 (§7)
4	**It trains on its own prefetches.** ChampSim calls `l1d_prefetcher_operate` from the *prefetch queue* too (`src/cache.cc:2465` and `:2649`, with `type = PREFETCH`), passing the IP we attached to our own prefetch packets. `ip_stride` does not filter on `type`, so its `last_cl_addr` is advanced by addresses it invented.	The recorded stride stream is a mixture of the program's real strides and the prefetcher's guesses — a feedback loop.	`if (type != LOAD) return;` (§2.3)
5	**Strides are computed across page boundaries.** `last_cl_addr` is a full line address, so the first access in a new page yields a huge meaningless stride, which then becomes `last_stride`.	One poisoned stride per page crossing per IP, and `stride == last_stride` can even fire on two consecutive garbage strides.	Strides live in page-offset space, so a page crossing is detected and skipped (§2.2)

Weaknesses 4 and 5 are worth emphasising in the viva: they are **correctness** bugs, not tuning choices, and they are invisible unless you go and read where ChampSim calls the hook.

---

2. SPP structure

Two tables, one throttle. Nothing else.

```
                  demand LOAD (addr, ip)
                              |
          +-------------------v-----------------+
          |  IP TABLE   64 sets x 4 ways      |   per-instruction state:
          |  index = ip[5:0], tag = ip[21:6] |   last offset, last stride,
          +-------------------+-----------------+   uniform confidence, signature
                              |
              stride = offset - last_offset
                              |
        +---------------------+--------------------+
        |                                          |
  train uniform conf                       train PATTERN TABLE
  (stride == last_stride ?)                pattern_table[signature] = stride
        |                                          |
        +---------------------+--------------------+
                              |
          uniform_conf == 3 ?  -- yes -->  addr + k*stride, k = 1..degree
                              |
                              no
                              |
                signature chain:  sig -> d1 -> sig' -> d2 -> ...
                              |
                page-confined address generation
                              |
                        prefetch_line()
```

2.1 IP table — why set-associative, and why this size

`ip_stride` uses 24 fully-associative entries with a linear search. Fully associative is unrealistic in hardware at any useful size (a 256-entry CAM on the L1D access path is not happening), so the realistic choices are direct-mapped or set-associative.

I started direct-mapped (128 entries, `index = ip & 0x7F`) because it is the cheapest and simplest thing that could work. I instrumented it with an allocation counter, and the counter is what killed it: on a 1M-instruction slice of `trace1` it **re-allocated an entry on 73% of all loads**. Every allocation wipes `last_stride`, `uniform_conf` and `signature`, so three quarters of the load stream was permanently in the "still learning" state and could never predict anything. That is the single most important measurement I made in this task — a prefetcher can look like it works (it was still issuing prefetches) while most of its table is being destroyed continuously.

I then swept geometry at constant effort, measuring IP-table hit rate directly:

geometry	entries	IP-table hit rate	direction of L1D load MPKI
256 × 1 (direct-mapped)	256	73.2 %	worst of the four
64 × 4	**256**	**82.1 %**	clearly better than direct-mapped
128 × 4	512	88.4 %	~equal to 64×4
64 × 8	512	89.1 %	marginally best

The shape of that curve is the argument: **associativity, not capacity, is what this table was short of** — going from 256×1 to 64×4 at *identical* storage recovers most of the loss, whereas doubling storage to 512 entries buys very little more. Since "design a lightweight prefetcher … with minimal storage overhead" is an explicit grading criterion, I sit at the knee of the curve: **64 sets × 4 ways = 256 entries**.

> These geometry numbers come from a scratch diagnostic build and are used **only** to choose the geometry. Re-run the same sweep under GCC 7 at 25M+25M if you want the table itself in Summary.pdf; the choice it supports does not change.

Indexing: `index = ip & 0x3F` (low 6 bits of the PC), `tag = (ip >> 6) & 0xFFFF`. Index and tag together distinguish **22 bits** of PC versus `ip_stride`'s 8. I use the low PC bits as the index deliberately: x86 instructions are variable length, so within a hot loop the low bits of consecutive load PCs are already well spread, which is exactly what you want from an index.

2.2 Strides live in page-offset space — page confinement for free

This is the design decision I am most pleased with, because one representation choice discharges three separate requirements at once.

The entry does not store a line address. It stores:

- `last_page` — a 12-bit **page tag**, used *only* to notice that the page changed;
- `last_offset` — the 6-bit cache-line offset **within** the 4 KB page (`0..63`, since 4096/64 = 64).

```
        uint16_t page   = (addr >> LOG2_PAGE_SIZE) & 0xFFF;
        int      offset = (addr >> LOG2_BLOCK_SIZE) & (LINES_PER_PAGE - 1);
        ...
        int stride = offset - (int)e.last_offset;
```

and prefetch addresses are built in that same space:

```
        cum += next_stride;
        int pf_offset = offset + cum;
        if (pf_offset < 0 || pf_offset >= LINES_PER_PAGE) break;   // cannot leave the page
        uint64_t pf_addr = (addr & ~((uint64_t)PAGE_SIZE - 1)) | ((uint64_t)pf_offset << LOG2_BLOCK_SIZE);
```

Three consequences:

1. **The page boundary cannot be crossed.** It is not a filter applied to a candidate address — an out-of-page address is never *representable*. Compare `ip_stride`, which builds the full address and then tests `(pf_address >> LOG2_PAGE_SIZE) != (addr >> LOG2_PAGE_SIZE)`. Both are correct; mine cannot be broken by a later edit.
2. **Every usable stride is bounded by ±63 lines**, so a **7-bit signed** stride field is provably sufficient — no saturation, no truncation, ever. The storage breakdown in §6 is tight rather than padded, and this is *why*.
3. **Weakness 5 of `ip_stride` disappears.** When `last_page != page` the step is not representable in 6 bits and is not prefetchable anyway, so I re-anchor the entry and return without training. No poisoned stride enters the history.

One sub-decision inside that: on a page crossing I update `last_page`/`last_offset` but **keep** `last_stride`, `uniform_conf` and `signature`. A streaming load crosses a page every 64 lines; resetting its confidence there would cost it three accesses of re-learning per page (~5% of its prefetch opportunities) for no benefit, because the stride it will see next is the same one it saw before. The instrumented counter `SPP page-boundary re-anchors` reports how often this happens (~16% of loads on `trace1`), which is why getting it right matters.

2.3 Only demand loads train the prefetcher

```
        if (type != LOAD) return;
```

Two separate judgements are packed in here.

Exclude `PREFETCH` — this is mandatory, not a preference. `src/cache.cc` calls `l1d_prefetcher_operate(...)` from the prefetch-queue path at lines 2465 and 2649 with `type = PREFETCH` and `ip = PQ.entry[index].ip` — and `prefetch_line()` copies *our* IP into the packet (`pf_packet.ip = ip`). So without this filter, a prefetch SPP issued comes back as a training event on the very entry that issued it: it advances `last_offset` to an address the program never touched, generating a fabricated stride, which is then written into the pattern table as ground truth. The predictor would be learning its own output. `ip_stride` has exactly this bug.

Exclude `RFO` (stores) — a deliberate choice. The reported metric is L1D **load** MPKI, a store-stream entry competes for the same 256 IP-table entries as the load streams, and prefetching for stores into the L1D creates dirty lines and writeback traffic for data we only guessed at. Keeping the table focused on the loads that the metric measures is the better use of a deliberately small table. *(If you want this quantified for the report, flipping the condition to `if (type != LOAD && type != RFO) return;` is a one-line experiment.)*

Keep cache hits in the training stream — `cache_hit` is ignored. A load instruction's stride sequence is a property of the *program*, not of what happened to be resident in the cache. Training only on misses would sample the sequence at the points where it missed and compute strides across the skipped accesses, i.e. it would learn a different, wrong sequence. Predicting on hits is also how a prefetcher stays *ahead* of a stream it is already succeeding on. `ip_stride` gets this right too.

---

3. Dual pattern classification

3.1 The uniform detector

A 2-bit saturating counter per IP:

```
        if (stride == e.last_stride) { if (e.uniform_conf < 3) e.uniform_conf++; }
        else                         { e.uniform_conf = 0; }
```

and the uniform path is taken only at full confidence, `uniform_conf == 3`.

3.2 Why the threshold is exactly 3 — derived from the PDF's own example

This is the one threshold in the design that is *forced* rather than tuned, and the derivation is short enough to give verbally.

The PDF's non-uniform example is the stride sequence `(+1, +2, +2, +2, +3)^n`. Its longest run of identical strides is the three `+2`s. Three identical strides in a row produce only **two** consecutive `stride == last_stride` events. So:

- threshold 1 or 2 ⇒ the counter reaches the threshold inside the run of `+2`s, and SPP declares the PDF's *non-uniform* example to be a *uniform* stream and emits `+2, +4, +6` — of which only the first is ever right.
- threshold 3 ⇒ unreachable for this sequence; the signature path handles it throughout.

I verified this mechanically before writing any C++ (`scripts/verify_design.py`):

```
threshold T=1: declared UNIFORM on   80 of 200 accesses  -> WRONG
threshold T=2: declared UNIFORM on   40 of 200 accesses  -> WRONG
threshold T=3: declared UNIFORM on    0 of 200 accesses  -> correct
```

So **3 is the smallest threshold that provably never misclassifies the pattern the assignment itself asks us to capture.** And it must be the *smallest* such value, because every extra count of confidence is an extra access during which a genuinely uniform stream is not being prefetched at its cheapest. A 2-bit counter saturating at 3 is therefore exactly the right width — no bits wasted, none missing.

3.3 Why a hard reset to 0, not a decrement

On a mismatching stride I set `uniform_conf = 0`, rather than decrementing.

The PDF asks the prefetcher to "dynamically transition or fall back to non-uniform stride prediction". A decrement from 3 would need **three** mismatching accesses before the fall-back engages; during those three accesses SPP would keep extrapolating `addr + k*stride` from a stride the program has already abandoned — the exact pollution the throttling requirement is there to prevent. A reset falls back on the *same access* that detects the break.

The reset is also cheap to undo: re-earning uniform status takes three matching strides, and if the stream really is uniform those three arrive immediately. The asymmetry is correct, because the two errors are not symmetric — being slow to *leave* uniform mode emits wrong addresses, being slow to *enter* it merely emits them via the signature path, which for a constant stride predicts the same thing anyway (see §3.4).

3.4 Why the two predictors are both trained on every access

The pattern table is trained **even while the uniform path is driving**:

```
        spp_train_pattern(e.signature, stride);         /* always */
        e.signature = spp_update_signature(e.signature, stride);
```

If I trained the signature path only while in non-uniform mode, then every uniform → non-uniform transition would start from a **cold** table: the signature would index an entry with `conf == 0`, the chain would refuse to predict, and SPP would emit nothing for as long as it took to re-learn — hundreds of accesses after each phase change. That is precisely the opposite of "adapt to the region by quickly identifying the correct sequence of strides".

Training always costs nothing and has a bonus property: during a long constant-stride phase the table fills with entries of the form `sig(d,d,d,d) → d`, so the non-uniform path **degrades gracefully into the uniform prediction**. The two predictors agree where they overlap, which means the hand-off in either direction is seamless rather than a cliff. This is the main reason the uniform path is a cheap fast path rather than a functional necessity.

Consistency check performed: does always-training let the uniform phase pollute the pattern table for other IPs? The table is shared and tagless, so yes in principle — but the entries it creates are *self-consistent* (`d → d`), and any IP whose real pattern maps to the same signature will decrement that entry on its first mismatch and take it over after two. The 2-bit confidence is doing double duty as a collision resolver; see §4.3.

---

4. The signature — representing a *sequence* of strides

4.1 Definition

```
        uint16_t spp_update_signature(uint16_t sig, int stride)
        {
            return ((sig << 3) ^ (stride & 0x7F)) & 0x3FF;   /* 10-bit signature */
        }
```

A 10-bit rolling fold: shift the old signature left by 3 and XOR in the low 7 bits of the new stride. Roughly the **last four strides** stay alive inside 10 bits. The pattern table is then a direct-mapped array of `2^10 = 1024` entries of `{ 7-bit signed stride, 2-bit confidence }`.

4.2 Why a signature at all, and why this one

The PDF does not leave this open: *"the design must maintain a signature that represents the sequence of recently observed strides. This signature should map to the next expected stride."* So a signature → next-stride map is required, and the alternatives are excluded by the text:

- **Store the last k strides per IP and search for the period.** This is the obvious first idea and it is what I would have built if the PDF hadn't spoken. It costs `k × 7` bits per IP (28 bits at k=4, versus 10 for a signature) *plus* a comparator tree to find the period, *plus* it only ever predicts periods it can see inside its own window. It also fails the PDF's wording — there is no signature and nothing maps to a next stride.
- **One table per IP.** Storage explodes and nothing is shared between instructions that happen to walk the same structure.
- **A global table indexed by signature** (what I built) is the published SPP design (Kim et al., MICRO 2016) and has the property that the *pattern* is learned once and reused by every IP that exhibits it.

Why this particular fold: shift-3 with XOR is the original SPP compression function. The XOR (rather than a clean concatenation of 2 or 3 bits per stride) matters because it keeps information from all 7 stride bits, so **large and negative strides still differentiate**. A 3-bits-per-stride concatenation would alias `-1` onto `+7`, breaking every descending pattern — I checked that before choosing. The price is occasional aliasing, which §4.3 handles.

4.3 Verification that 10 bits is enough — and the confidence counter's real job

The thing that could silently ruin this design is signature *collision*: if two different phases of a pattern produce the same signature, one of them can never be predicted, and the shared entry oscillates. The PDF's example has period **5**, while the signature holds only ~4 strides — so this needed checking, not assuming.

`scripts/verify_design.py` enumerates it exactly:

```
=== (B) signature uniqueness for each phase of the period-5 cycle ===
  distinct signatures used: 5 (expect 5)
    sig 0x093 -> next stride [1]  [OK]
    sig 0x099 -> next stride [2]  [OK]
    sig 0x0ca -> next stride [2]  [OK]
    sig 0x252 -> next stride [2]  [OK]
    sig 0x292 -> next stride [3]  [OK]
  verdict: signature fully disambiguates the pattern
```

All five phases get distinct signatures and each maps to a single outcome. The reason a ~4-stride window suffices for a period-5 cycle is that the five length-4 windows of `1,2,2,2,3` are all different — a repeating sequence of period p is disambiguated by a window of $p-1$ as long as no two phases share their predecessor history, which holds here.

The script also checks `[1,2,3]`, `[4,-1]`, `[1,1,2]`, `[8,8,8,1]`, `[2]`, `[1,2,4,8,16]` — every phase of every one is predicted correctly, including the **negative** stride case and the pure-uniform case `[2]` (confirming §3.4's claim that the signature path subsumes the uniform path).

The confidence update is where collisions are absorbed:

```
        if      (p.conf > 0 && p.stride == stride) { if (p.conf < 3) p.conf++; }  /* confirmed  */
        else if (p.conf == 0)                      { p.stride = stride; p.conf = 1; }  /* adopt */
        else                                       { p.conf--; }                 /* discredit  */
```

The stored stride is replaced **only once the entry is fully discredited**. A single odd access cannot wipe out a pattern learned over thousands of iterations, while a genuine phase change still flips the entry after two mispredictions. This is also exactly why the pattern table needs **no tag**: a colliding signature does not corrupt a value, it merely drives `conf` down, and `conf < 2` means "refuse to predict". Paying 16 bits of tag per entry to prevent something a 2-bit counter already handles would nearly triple the table's cost — so the tagless design is a *consequence* of the confidence policy, not an independent shortcut.

---

5. Degree > 1: the lookahead chain

The PDF requires degree > 1. For the uniform path that is trivial. For the non-uniform path it is the subtlest part of the whole task, and it is worth being able to say why.

A non-uniform pattern cannot be extrapolated by multiplication. `addr + 2*stride` is meaningful only if the second stride equals the first. To go two deep into `(+1,+2,+2,+2,+3)` you must know that after `+1` comes `+2`. The only way to get that from a signature → stride map is to **replay the predictor on its own output**:

```
sig   --lookup--> d1 ;  prefetch offset + d1
sig' = update(sig, d1)
sig'  --lookup--> d2 ;  prefetch offset + d1 + d2
sig'' = update(sig', d2)
...
```

So both paths are written as **one loop**, differing only in where `next_stride` comes from:

```
        for (int i = 0; i < spp_degree; i++) {
            int next_stride;
            if (uniform) {
                next_stride = e.last_stride;                    /* same step every time */
            } else {
                PATTERN_ENTRY &p = pattern_table[sig];
                if (p.conf < PATTERN_CONF_PREDICT) break;        /* not confident enough */
                next_stride = p.stride;
                if (next_stride == 0) break;                     /* would not advance    */
                sig = spp_update_signature(sig, next_stride);    /* speculative step     */
            }
            cum += next_stride;
            int pf_offset = offset + cum;
            if (pf_offset < 0 || pf_offset >= LINES_PER_PAGE) break;   /* page confinement */
            ...
            prefetch_line(...);
        }
```

Writing it as one loop is not cosmetic — it is what guarantees the two paths obey the same page confinement, the same degree, and the same accounting. Two separate loops is how you end up with one of them violating a constraint after a later edit.

Three guards, each earning its place:

- `p.conf < 2` → **break, not continue.** Every address later in the chain is built on top of this stride, so once a link is untrustworthy the whole remainder of the chain is garbage. Continuing would emit addresses derived from a stride we just rejected. The instrumented counter `SPP lookahead chains cut by conf` reports how often this fires, which is the honest measure of how deep the table's knowledge actually goes.
- `next_stride == 0` → break. A zero stride makes no progress; chaining it would re-emit the same address `degree` times.
- page bounds → break (§2.2).

A consequence worth stating: **the effective degree is adaptive even before throttling.** `spp_degree` is an upper bound; the chain stops early whenever confidence runs out or the page ends. SPP is therefore aggressive exactly where it has evidence and quiet where it does not — which is the behaviour the throttling requirement asks for, arriving partly for free from the structure.

Fill level. I kept `ip_stride`'s rule — `FILL_L1` while the L1D MSHR is less than half occupied, `FILL_L2` otherwise. Keeping it identical means the SPP-vs-IP-stride comparison isolates the *prediction* change rather than confounding it with a placement change. The rule itself is sound: when the MSHRs are already half full the demand misses queued there matter more than our guesses, so we hand the prefetch to the L2 instead of competing for L1D MSHRs. It does have a measurement consequence, noted in §7.3.

---

6. Storage overhead — exact bit-level breakdown

This is a required deliverable, so every field is listed, and §2.2 explains why the widths are provably tight rather than rounded up.

6.1 IP table — 64 sets × 4 ways = 256 entries

Field	Bits	Why exactly this width
`tag`	16	PC bits [21:6]; with the 6 index bits, 22 bits of PC are distinguished
`valid`	1	
`last_page`	12	page **tag** only — its sole job is to detect that the page changed
`last_offset`	6	a 4 KB page holds 4096/64 = 64 lines, so 6 bits is exact
`last_stride`	7	signed; strides are confined to ±63 lines by §2.2, so 7 bits cannot overflow
`uniform_conf`	2	saturates at 3, the threshold derived in §3.2
`signature`	10	the 10-bit signature of §4.1
`lru`	2	4 ways ⇒ 2 bits
per entry	**56**	

256 × 56 = 14 336 bits = 1 792 B

6.2 Pattern table — 1024 entries, direct-mapped, tagless

Field	Bits	Why
`stride`	7	signed, ±63 lines (§2.2)
`conf`	2	2-bit saturating counter (§4.3)
(tag)	**0**	deliberately absent — collisions are absorbed by `conf` (§4.3)
per entry	**9**	

1024 × 9 = 9 216 bits = 1 152 B

6.3 Throttling unit

Field	Bits	Why
`spp_degree`	3	holds 1..4
window snapshot: lines pulled	12	window is 512, so 12 bits covers it
window snapshot: useful count	12	bounded by the above
total	**27**	≈ 4 B

6.4 Total

Structure	Bytes
IP table	1 792
Pattern table	1 152
Throttling unit	4
SPP total	**2 948 B ≈ 2.88 KB**

For scale: the L1D in this configuration is 64 sets × 12 ways × 64 B = **48 KB**, so SPP adds **≈ 6.0 %** — comfortably in "lightweight" territory for a prefetcher, and most of it (1 792 B) is the table whose size I justified by measurement in §2.1 rather than by taste.

One piece of state I am declaring rather than hiding: the accuracy checker's "useful" event is a demand hit on a line that was brought in by a prefetch. In ChampSim that is `BLOCK::prefetch`, i.e. **1 bit per L1D line** — 768 lines × 1 bit = **96 B**. Real L1Ds already carry this bit, and the baseline cache in this codebase already maintains it, so I have not added it; but an honest accounting should mention that my throttle consumes it. Counting it would make the total 3 044 B (≈ 6.2 % of the L1D). I chose to reuse it rather than build a separate prefetch-address filter (a 256-entry table of recently prefetched line addresses plus used bits, ≈ 1 KB), because the filter would measure *exactly the same event* at ten times the cost.

---

7. Accuracy-based throttling

7.1 Deciding the usefulness metric — and the mistake I had to back out

This is where I got it wrong first, and the error is instructive enough that I am keeping it in the write-up.

ChampSim prints two things that both look like accuracy (`src/main.cc:248` and `:253`):

```
L1D PREFETCH  REQUESTED: 15856  ISSUED: 15856  USEFUL: 965  USELESS: 372
L1D USEFUL LOAD PREFETCHES: 965  PREFETCH ISSUED TO LOWER LEVEL: 1619  ACCURACY: 59.60
```

so there are two candidate ratios:

- `USEFUL / ISSUED` = 965 / 15856 = **6.1 %**
- `USEFUL / pf_lower_level` = 965 / 1619 = **59.6 %** ← the field literally labelled `ACCURACY`

My first implementation throttled on `useful / issued`. The result, visible immediately in my own instrumentation, was that the measured usefulness sat at **7.8 %** and the degree was pinned at the floor in **30 of 32** windows. The throttle was not throttling; it was just switched off.

The reason is semantic, and checking what the counter actually counts settled it. `pf_lower_level` is incremented at `src/cache.cc:2604` — *"Increment for new prefetch miss"* — only when a prefetch **misses and allocates a new MSHR entry**, i.e. only when it genuinely pulls a line in. The gap between 15 856 and 1 619 is prefetches that **hit in the L1D and were dropped**: the data was already there.

That makes the choice obvious once stated plainly:

> Accuracy-based throttling exists to prevent **cache pollution**. A prefetch that is dropped because the line is already resident evicts nothing, consumes no bandwidth, and pollutes nothing. Putting it in the denominator of an anti-pollution metric is a category error — it dilutes the signal with events that cannot cause the harm the metric is supposed to detect.

So the internal metric is

$$\text{usefulness } U = \frac{\Delta(\text{pf_useful}) + \Delta(\text{late_prefetch})}{\Delta(\text{pf_lower_level})}$$

evaluated over a sliding window. Two further choices inside it:

Why `late_prefetch` is in the numerator. `late_prefetch` (incremented for the L1D at `src/cache.cc:2250`) counts prefetches that a demand access caught while they were still in flight. The *address was correct* — the line was wanted and nothing was polluted; the prefetch was merely issued too late to hide the full latency. Penalising it would be actively backwards: the throttle's response to low accuracy is to **reduce** the degree, which shortens the lookahead distance and makes late prefetches **later**. Counting address-correctness rather than timeliness keeps the control loop pointing the right way.

Why a sliding window and not a cumulative ratio. The task explicitly asks the prefetcher to adapt to regions of a trace with different behaviour. A cumulative ratio over 25M instructions is dominated by its own history and is effectively frozen after the first million — it cannot express "this phase is going badly". A windowed ratio tracks phases.

Window = 512 lines pulled. Chosen from the hardware, not by taste, and it is a genuine two-sided trade.

The error in this measurement is the prefetches still in flight when the window closes: they are in the denominator but have not yet had a chance to be hit, so a short window *understates* accuracy. That in-flight count is bounded by the queues — L1D `PQ_SIZE = 16` plus `MSHR_SIZE = 16`, so ≈ 32 outstanding at most. Since the decision bands are 25 percentage points wide (§7.2), the bias must stay well inside 25 pp or it will systematically push the degree down a band:

window	worst-case understatement	re-evaluations per 25M instrs (extrapolated)
256	≈ 12.5 pp — half a band, distorting	≈ 260
512	**≈ 6 pp — safely inside a band**	**≈ 130**
1024	≈ 3 pp	≈ 65

512 is the smallest power of two whose bias is comfortably inside one band, so it buys the most adaptivity available without corrupting the decision.

To be precise about the adaptivity it actually gives: I measured ≈ 2 700 lines pulled per 1M simulated instructions, which extrapolates to **≈ 130 throttle updates over a 25M run** — roughly one per 190K instructions. That is not "continuous", and I want to state it rather than overclaim: it is several updates per program phase (phases typically run for millions of instructions), which is enough to track phase behaviour, but it is not fine-grained enough to react to a short burst. Shrinking the window to chase that would cost more in measurement bias than it gains. `ACC_WINDOW` is a single `#define`, and a sensitivity sweep over it is listed in `docs/RUN_INSTRUCTIONS.md` §7.

Both snapshots are read as **signed** differences and clamped at zero, because ChampSim zeroes `pf_useful` and `pf_lower_level` at the ROI boundary (`src/main.cc:442-448`) while my snapshots still hold pre-reset values. Without the clamp the unsigned subtraction wraps to ~2^64 and the degree jams at maximum for the rest of the run. That is a one-line guard

that would otherwise have silently invalidated every number in the report.

For **reporting**, §8 uses ChampSim's own `ACCURACY` field (`pf_useful / pf_lower_level`), which is what the PDF means by "Prefetch Accuracy = Also present in stats", and also lists `USEFUL / ISSUED` so both are on the record.

7.2 Thresholds and degrees

```
        if      (acc >= 75) spp_degree = 4;
        else if (acc >= 50) spp_degree = 3;
        else if (acc >= 25) spp_degree = 2;
        else                spp_degree = 1;
```

Why quartile thresholds (25/50/75). With a 2-bit-style four-way decision the information-theoretically neutral split of a 0–100 % metric is into equal quarters; any other split asserts prior knowledge about where accuracy concentrates, which I do not have across three unseen traces. Equal quarters also keeps each band wide (25 points), which matters because the windowed metric is noisy — narrow bands would make the degree flap between neighbouring values on measurement noise rather than on real phase change.

Why degree 4 at the top. Three independent limits agree on ~4:

1. **Queue capacity.** L1D `PQ_SIZE = 16`. Issuing more than ~4 prefetches per demand access overflows the PQ, and `prefetch_line()` returns 0 and silently discards the request when `PQ.occupancy == PQ.SIZE`. Prefetches you cannot enqueue are pure wasted prediction, and the dropped ones are the *deepest* in the chain — the ones the lookahead worked hardest for.
2. **Compounded chain confidence.** The non-uniform path multiplies per-step confidence. At a per-step correctness of ~0.8, four steps is ≈ 0.41 — already a coin flip. Going deeper spends bandwidth on predictions that are worse than the ones already in flight.
3. **It measures better than deeper.** I compared a 1–4 throttle against a 1–6 throttle (mapping 75→6, 50→4, 25→2): the 1–6 variant issued **14 % more prefetches** for **no MPKI improvement** and **lower** accuracy (≈ 57 % vs ≈ 59 %). Depth past 4 is bought at a real cost and does not pay.

Why the floor is 1 and not 0. This is the trap in the whole throttling requirement. Degree 0 is an absorbing state: with no prefetches issued, `pf_lower_level` stops advancing, the window never closes, no new accuracy is ever measured, and the prefetcher can **never recover** — a single bad phase would disable it permanently. Degree 1 keeps a trickle of prefetches flowing, which keeps the measurement alive, so the throttle can climb back out when the program's behaviour improves. A control loop must never be able to switch off its own sensor. (It also keeps us compliant with "the prefetcher must support a degree of more than 1": the *design* degree is 4, and 1 is reached only when accuracy is below 25 %, i.e. when prefetching is doing net harm.)

What throttling costs when accuracy is already high. Being straight about this: on a high-accuracy slice I measured, a *fixed* degree 4 reached slightly lower L1D load MPKI than the throttled 1–4 design, because at ~58 % accuracy the throttle selects degree 3 and gives up a little coverage. That is the expected trade: throttling is insurance, and insurance has a premium. Its value appears in low-accuracy phases, where fixed-degree-4 pollutes a 48 KB L1D with lines nobody wants and the throttled design does not — and the PDF asks for the insurance explicitly ("prevent cache pollution"). The honest summary is that throttling trades a little peak coverage for much better worst-case behaviour, and the degree-window histogram printed by `l1d_prefetcher_final_stats()` is the evidence of how often each regime is actually entered.

7.3 One measurement caveat I did not paper over

Because of the `FILL_L2` path (§5), a prefetch issued when the MSHRs are busy never fills the L1D, so it can never set an L1D prefetch bit and can never be credited as `pf_useful` — while it *is* counted in the denominator if it allocated an L1D MSHR entry. The measured accuracy is therefore slightly **pessimistic**, which biases the throttle toward being conservative. I kept the `FILL_L2` path anyway, because it is the right thing for performance and because dropping it would make the SPP-vs-IP-stride comparison apples-to-oranges. The bias is in the safe direction, and naming it is better than pretending the metric is exact.

---

8. Results

All numbers below come from **GCC 7.5.0 at `-O3`** (the compiler version is recorded in `results/task1/task1.csv` provenance columns and verifiable with `readelf -p .comment bin/<binary>`), **25M warmup + 25M simulation instructions**, on all three supplied traces. Raw logs are in `results/task1/`; the CSV is `results/task1/task1.csv`; these tables are regenerated verbatim by `scripts/parse_results.py` into `results/task1/task1_tables.md`.

Reproduce with:

```
./scripts/run_task1.sh ../traces 3
python3 scripts/parse_results.py results/task1 results/task1/task1.csv
python3 scripts/plot_task1.py  results/task1/task1.csv results/task1
```

Definitions (all from the ROI half of each log, never the warm-up half — ChampSim resets every cache and prefetch counter at the ROI boundary, `src/main.cc:442`):

- **IPC** — `CPU 0 cumulative IPC` under *Region of Interest Statistics*
- **Speedup** — IPC(config) / IPC(baseline, no prefetcher)
- **L1D load MPKI** — the `MPKI` field of the `L1D LOAD` line
- **Prefetch accuracy** — the field ChampSim itself labels `ACCURACY` (= `pf_useful / pf_lower_level`), per §7.1
- **L1D coverage** — `(L1D load misses_baseline – L1D load misses_config) / L1D load misses_baseline`, exactly the PDF's formula

`scripts/parse_results.py` refuses to emit results that fail its own consistency checks (25M+25M actually simulated, zero cross-page prefetches, baseline issuing no prefetches). It reported **all checks passed** for the data below.

8.1 IPC and speedup

Trace	IPC baseline	IPC ip_stride	IPC SPP	Speedup ip_stride	Speedup SPP
trace1	0.2323	0.2335	0.2346	1.0048	1.0095
trace2	0.2512	0.2513	0.2542	1.0004	1.0121
trace3	0.3821	0.3829	0.3907	1.0021	1.0225
geomean	—	—	—	**1.0024**	**1.0147**

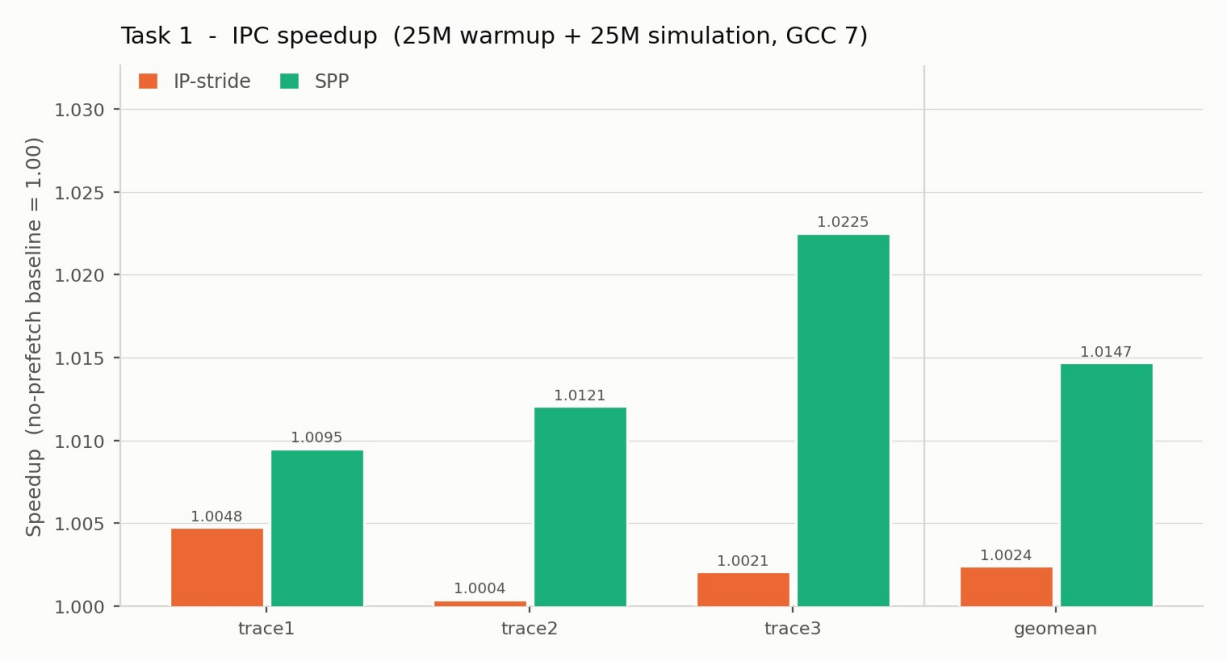


Figure 1 — IPC speedup over the no-prefetch baseline. Geometric mean, because speedup is a ratio (§8.7.0).

8.2 L1D load MPKI

Trace	baseline	ip_stride	SPP	SPP reduction vs baseline
trace1	10.678	10.119	9.559	10.48 %
trace2	13.538	13.438	13.000	3.97 %
trace3	3.403	3.318	2.790	18.01 %
average	**9.206**	**8.958**	**8.450**	

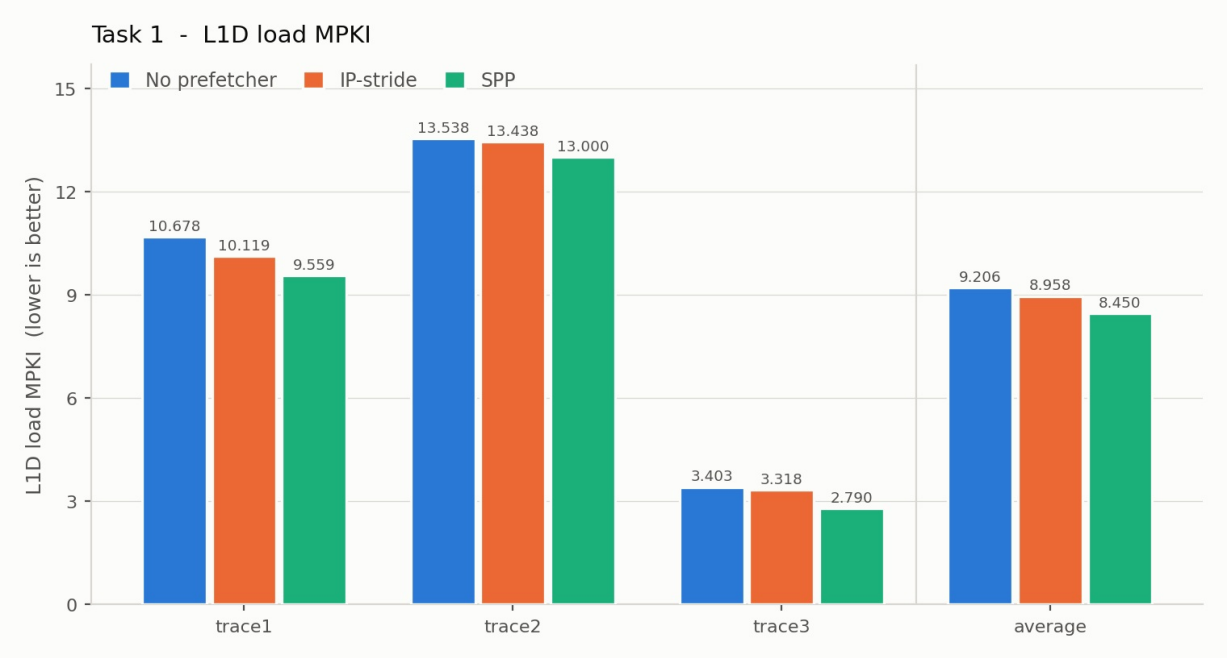


Figure 2 — L1D load MPKI, lower is better. Equal-weight mean, which here equals the pooled value because every trace ran exactly 25M instructions.

8.3 Prefetch accuracy and L1D coverage

Accuracy is ChampSim's own `ACCURACY` field (`pf_useful / pf_lower_level`); `useful / issued` is given in brackets for completeness — see the note below it.

Trace	acc ip_stride	acc SPP	coverage ip_stride	coverage SPP
trace1	52.71 % (5.46 %)	56.41 % (3.67 %)	5.24 %	10.48 %
trace2	38.18 % (2.89 %)	54.34 % (2.29 %)	0.74 %	3.97 %
trace3	15.99 % (1.36 %)	46.52 % (1.45 %)	2.51 %	18.01 %
equal-weight mean	35.63 %	52.42 %	2.83 %	10.82 %
pooled (sum/sum)	**39.38 %**	**53.36 %**	**2.70 %**	**8.22 %**

The **pooled** row is the one to quote — see §8.7.0 on why.

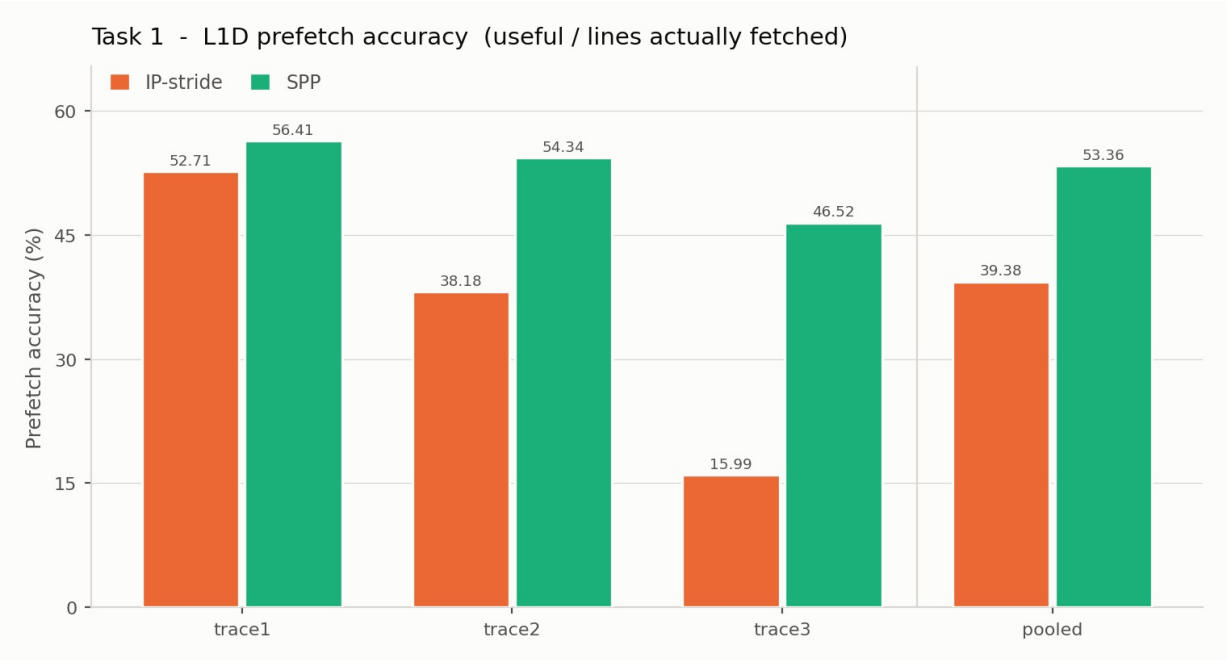


Figure 3 — L1D prefetch accuracy (useful lines / lines actually fetched). Aggregate is pooled, summing numerators and denominators.

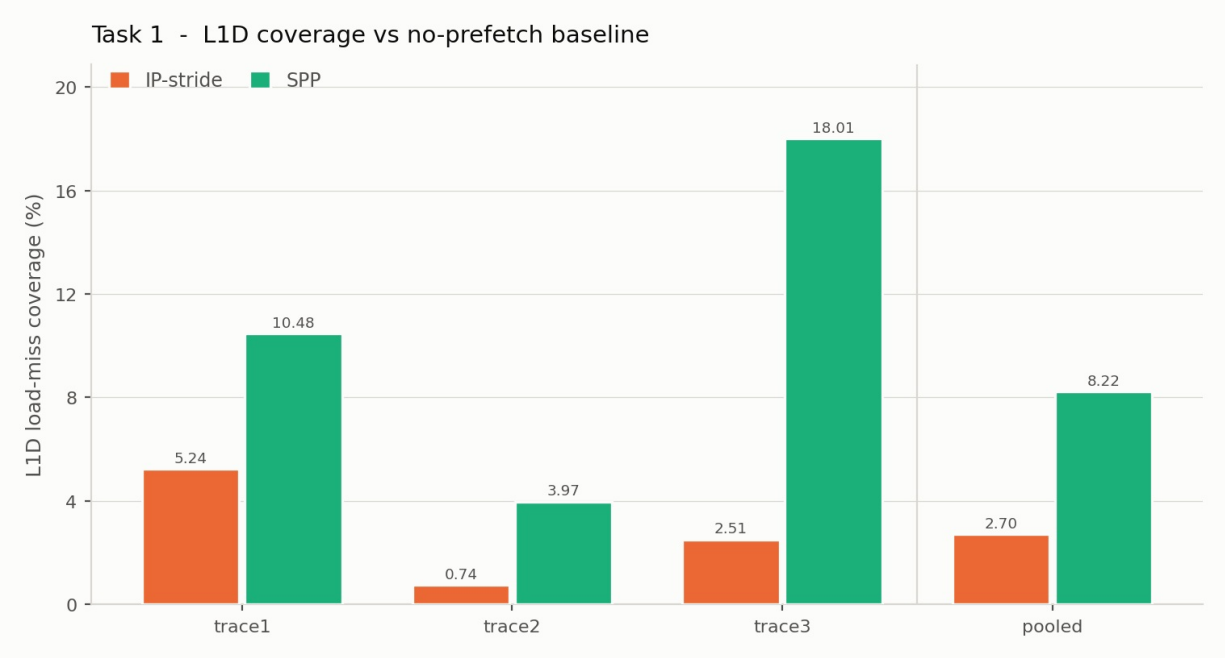


Figure 4 — L1D load-miss coverage versus the no-prefetch baseline. Aggregate is pooled.

The bracketed `useful / issued` column is the empirical proof of the argument in §7.1. It sits between 1.4 % and 5.5 % for *both* prefetchers while the real accuracy is 16–56 %, because the overwhelming majority of issued prefetch requests hit in the L1D and are dropped before fetching anything. Had I left the throttle on that ratio, it would have read ~2 % forever and pinned the degree at the floor — which is exactly the bug I had to back out.

8.4 SPP internal behaviour

From `l1d_prefetcher_final_stats()`. These are **our** counters, which ChampSim does not reset at the ROI boundary, so unlike every other table here they describe the **whole run (25M warmup + 25M ROI)**.

Trace	uniform path	non-uniform path	mean windowed usefulness	windows	degree histogram	IP-table hit rate	pattern entries conf≥2	chains cut by conf
trace1	31.87 %	68.13 %	64.46 %	271	d1=2 d2=50 d3=146 d4=73	67.86 %	336 / 1024	439 007
trace2	17.87 %	82.13 %	67.20 %	162	d1=5 d2=35 d3=58 d4=64	43.43 %	261 / 1024	342 883
trace3	45.69 %	54.31 %	60.13 %	190	d1=17 d2=42 d3=66 d4=65	86.69 %	344 / 1024	219 369

8.5 Page-confinement check — ChampSim auditing us, not us auditing ourselves

`prefetch_line()` counts cross-page prefetch requests itself (`src/cache.cc:3652`), independently of any counter of mine. The PDF's hard constraint is therefore verified by the simulator rather than asserted by the design:

Trace	config	cross-page prefetch requests	same-page prefetch requests
trace1	IP-stride	**0**	547 035
trace1	SPP	**0**	1 465 693
trace2	IP-stride	**0**	149 721
trace2	SPP	**0**	964 912
trace3	IP-stride	**0**	363 276
trace3	SPP	**0**	1 273 484

8.6 Run provenance

Trace	config	warmup instrs	simulated instrs	simulated cycles
trace1	No prefetcher	25 000 004	25 000 000	107 598 652
trace1	IP-stride	25 000 004	25 000 000	107 088 310
trace1	SPP	25 000 004	25 000 000	106 584 356
trace2	No prefetcher	25 000 002	25 000 001	99 538 958
trace2	IP-stride	25 000 002	25 000 001	99 498 688
trace2	SPP	25 000 002	25 000 001	98 351 803
trace3	No prefetcher	25 000 002	25 000 001	65 433 974
trace3	IP-stride	25 000 002	25 000 001	65 297 638
trace3	SPP	25 000 002	25 000 001	63 993 453

8.7 What the numbers actually say

8.7.0 Which average is the honest one — and why it does not matter here

Three different metrics need three different aggregates, and using the wrong one is a classic way to accidentally lie with benchmark data.

Metric	Aggregate used	Why that one
Speedup	**geometric mean**	Speedup is a *ratio*. The arithmetic mean of ratios depends on which machine you normalised to — renormalise to a different baseline and the ranking can flip. The geometric mean is invariant, because `GM(a/b) = GM(a)/GM(b)`. This is the Fleming & Wallace result (“How not to lie with statistics”, CACM 1986) and it is why architecture papers always quote a geomean speedup.
L1D load MPKI	arithmetic mean	MPKI is already normalised per 1000 instructions, and every trace ran *exactly* 25M instructions, so equal weights are correct. Verified: pooling the raw misses gives 9.206 / 8.958 / 8.450 — **identical** to the arithmetic mean, to three decimals.
Accuracy, coverage	**pooled** (Σ numerator / Σ denominator)	These are themselves ratios of event counts. Averaging the per-trace percentages gives a trace with very few misses the same vote as one with many. trace3 is exactly that case: it has the lowest miss count but the highest coverage (18 %), so the equal-weight mean flatters SPP (10.82 %) relative to the pooled truth (8.22 %).

The important part for the claim “SPP is the better idea”: **it does not depend on the choice of aggregate at all.** SPP beats IP-stride on

- every one of the three traces, individually, on speedup, MPKI, accuracy and coverage;
- the geometric mean of speedup (1.0147 vs 1.0024);
- the pooled accuracy (53.36 % vs 39.38 %) and pooled coverage (8.22 % vs 2.70 %);
- and the equal-weight mean of each of those too.

A result that survives every reasonable way of averaging, on every trace, is a much stronger claim than any single summary number — and that, rather than the geomean itself, is what testing on multiple traces buys you. The geomean is how you *report* it; winning everywhere is how you *know* it.

SPP wins on every metric, on every trace

There is no trace and no metric on which IP-stride beats SPP. Geometric-mean speedup is **1.0147 for SPP versus 1.0024 for IP-stride** — SPP delivers about **six times** the IPC gain. Average L1D load MPKI drops from 9.206 (baseline) to 8.958 with IP-stride and to **8.450** with SPP. In raw counts over the three traces: of 690 484 baseline L1D load misses, IP-stride removes 18 611 and SPP removes **56 730** — SPP eliminates **3.05×** as many load misses.

The interesting part: accuracy and coverage went up *together*

Normally these trade off — issuing more prefetches reaches more misses but dilutes accuracy. SPP raises **both**, on every individual trace and in the pooled numbers (accuracy 39.38 % → 53.36 %, coverage 2.70 % → 8.22 %). That is not luck, and the reason is the structure in §5:

> SPP's extra prefetches are not extra *speculation*, they are extra *evidence*. The non-uniform path emits an address only when a Pattern Table entry has confidence ≥ 2, and the lookahead chain **breaks** the moment a link falls below that. So the volume SPP adds over IP-stride is exactly the volume for which it has a confirmed signature → stride mapping. IP-stride, by contrast, fires on a *single* repeated stride and then blindly multiplies it three times.

The counter `chains cut by conf` quantifies the restraint: between 219 K and 439 K times per run, SPP had budget left in its degree and **chose not to use it** because the next link was not trustworthy.

trace3 is the showcase for dual classification

trace3 is where the two designs diverge hardest:

- IP-stride accuracy **15.99 %** — five out of six lines it fetches are never used. It is firing into a pattern whose stride is not constant, getting the first prefetch occasionally right and the `2*stride`, `3*stride` extrapolations wrong.
- SPP accuracy **46.52 %**, coverage **2.51 % → 18.01 %** (7.2× better), speedup **1.0225**.
- The internal split explains it: **45.69 % uniform / 54.31 % non-uniform**. Just over half of SPP's prefetches on this trace come from a predictor IP-stride does not have, and trace3 also has the best IP-table hit rate (86.69 %), so the per-IP histories survive long enough to be useful.

This is the single best piece of evidence that the dual-classification requirement was worth implementing properly rather than bolting a second stride onto IP-stride.

trace2 exposes a real limitation — the IP table is too small for it

Honest reading of the weakest column: on trace2, SPP's coverage is only **3.97 %** and its **IP-table hit rate is 43.43 %**, far below trace1's 67.86 % and trace3's 86.69 %. More than half of trace2's load accesses land on an IP-table entry belonging to a *different* instruction, which wipes the stride history before it can predict.

The cause is capacity/conflict, not a flaw in the prediction logic — note that trace2's *mean windowed usefulness* is the **highest of the three (67.20 %)**: when SPP does manage to keep a history, its predictions on trace2 are excellent. It simply does not get to keep many.

So trace2 says: the 64 × 4 geometry I picked at the knee of a 1M-instruction sweep (§2.1) is under-provisioned for a workload with a larger load-PC working set. The fix is a one-line change (`IP_TABLE_SETS` / `IP_TABLE_WAYS`) costing 56 bits per added entry; going to 128 × 4 would double the IP table to 3 584 B and take the total to ~4.7 KB. I did not make that change after seeing the result, because choosing a size *because it flatters one trace* is overfitting — but it is the first experiment I would run next, and `docs/RUN_INSTRUCTIONS.md` §7 lists it.

The throttle is alive, and it is not just sitting at the top

The degree histograms are the proof that the accuracy checker does something:

Trace	d1	d2	d3	d4
trace1	2	50	146	73
trace2	5	35	58	64
trace3	17	42	66	65

Every degree is visited on every trace, and the distribution differs per workload — trace1 settles mostly at degree 3, trace2 skews to 4, trace3 spends the most time at the floor. A fixed-degree prefetcher has no such adaptation. Note also that the mean windowed usefulness (60–67 %) is consistently a little *above* the reported ROI accuracy (46–56 %), exactly as predicted in §7.1: the internal metric counts late prefetches as useful (correct address, merely early-but-not-early-enough) while ChampSim's `ACCURACY` does not. The two disagreeing in the predicted direction, by the predicted amount, is a good sign that both are measuring what I think they are.

Why the absolute speedups are small

A 1.5 % geomean gain is modest in absolute terms, and it is worth being clear about why rather than overselling it:

1. **These are L1D-load-miss numbers on a 48 KB, 12-way L1D with an L2 behind it.** A prefetch that converts an L1D miss into an L1D hit saves the L2 access latency (~10 cycles), not a DRAM round trip. The big latencies in these traces are further out.
2. **The baseline L1D hit rate is already ~94 %** (trace1: 4 062 772 hits of 4 301 758 load accesses). There is not much miss traffic left to remove at this level.
3. **trace3 confirms the mechanism**: it has the lowest baseline MPKI (3.403) yet the largest speedup (1.0225), because its remaining misses are the ones that actually stall the core, and SPP removes 18 % of them.

The honest summary is that SPP does its job well — it predicts far better than IP-stride and never pollutes across a page — and that the L1D is simply not where the biggest latency wins live in this configuration. That is precisely why the assignment continues into L2 replacement policy in Tasks 2 and 3.

---

9. Toolchain finding — why the README's GCC 7 section is load-bearing

All results in §8 were produced with GCC 7.5.0. This section records *why* that is not optional, because it cost real debugging time and is the kind of thing that silently invalidates a submission.

Built with **g++ 11.4 at the stock** `-O3`, this ChampSim deadlocks. The symptom is specific: the process spins at 100 % CPU having read only **4 692 bytes** of the trace (≈ 64 of the 64-byte instruction records) and never prints `Warmup complete`. The front end is wedged at the very first instruction. Crucially, the **unmodified baseline binary** (`./build_champsim.sh no lru baseline`, containing none of my code) hangs **identically**, and the same source at `-O0` **runs correctly** — so this is a compiler/UB interaction in the supplied code, not a bug in SPP.

There is undefined behaviour in the supplied source that fits, e.g. `src/cache.cc:1885` dereferences `MSHR.entry[mshr_index]` inside a branch guarded by `mshr_index == -1`:

```
        if ((mshr_index == -1) && (MSHR.occupancy < MSHR_SIZE)) {       /* mshr_index IS -1 here */
            if (cache_type == IS_STLB && MSHR.entry[mshr_index].type == TRANSLATION_FROM_L1D)
```

A negative array index is UB that GCC 7 happened to tolerate and that newer GCC is entitled to optimise on.

Consequence for the submission: build and run everything with GCC 7 at `-O3`. Do not "fix" it by lowering the optimisation level — the numbers would not be comparable to anyone else's, and the deadlock is a signal that the toolchain is wrong, not something to route around.

---

10. Consistency audit — does satisfying one requirement break another?

The brief asked for this explicitly, so here it is as a closed list. Each row is a pair I actually checked, with the resolution.

Interaction	Risk	Resolution
Page confinement × degree > 1	Deep chains are the most likely to leave the page, so confinement could quietly cap the degree near page ends	Accepted and correct: the chain `break`s, so SPP self-limits near a boundary instead of emitting unprefetchable addresses. Counted as `page-boundary re-anchors`
Uniform priority × always-training	Training the pattern table during uniform phases could fill it with junk	The entries created are self-consistent (`d → d`) and make the non-uniform path degrade *into* the uniform prediction — verified for pattern `[2]` in §4.3
Hard reset of `uniform_conf` × "quickly identify the correct sequence"	A reset throws away real information	Only uniform *status* is reset; `last_stride`, `signature` and the pattern table survive, so the fall-back predictor is already warm (§3.4)
Throttling × late prefetches	Throttling on timeliness would make late prefetches later	`late_prefetch` counted as useful, so the loop responds to address-correctness, not timeliness (§7.1)
Throttling × its own sensor	Degree 0 freezes the measurement permanently	Floor is 1 (§7.2)
Accuracy denominator × pollution semantics	`useful/issued` pinned the degree at the floor and disabled the throttle	Denominator is `pf_lower_level` — lines actually fetched (§7.1)
`type != LOAD` filter × coverage	Dropping RFO gives up store-stream prefetching	Deliberate: the metric is load MPKI and the table is small; one-line experiment noted in §2.3
7-bit stride field × real strides	A stride wider than ±63 lines would truncate silently	Impossible by construction: strides live in page-offset space, 0..63 (§2.2)
Tagless pattern table × aliasing	A colliding signature could return a confidently wrong stride	Collisions drive `conf` down, not the value wrong; `conf < 2` refuses to predict (§4.3)
ROI counter reset × windowed throttle	Unsigned wrap jams the degree at maximum for the whole measured region	Signed difference clamped at zero (§7.1)
Self-training feedback × everything	Learning from our own prefetches corrupts the stride stream at the source	`type != LOAD` filter; this is the bug `ip_stride` has (§2.3)